Nama : Laurentius Noval Raditya Pratama

NIM : 2415344006

Kelas : 5B Teknik Otomasi

Code main.py

```
# Mission-06D - KNN pada ESP32 (MicroPython)
import time
import gc

# temperature (C), vibration (mm/s), label (0=NORMAL; 1=WARNING)
TRAIN = [
    (29.0, 1.0, 0),
    (42.4, 2.7, 0),
    (55.8, 4.4, 0),
    (68.0, 6.1, 1),
    (31.4, 7.8, 0),
    (44.8, 1.2, 0),
    (57.0, 2.9, 0),
    (70.4, 4.6, 1),
    (33.8, 6.3, 0),
    (46.0, 8.0, 1),
    (59.4, 1.4, 0),
    (72.8, 3.1, 1),
    (35.0, 4.8, 0),
    (48.4, 6.5, 1),
    (61.8, 8.2, 1),
    (74.0, 1.6, 0),
    (37.4, 3.3, 0),
    (50.8, 5.0, 0),
    (63.0, 6.7, 1),
```

```
    (76.4, 8.4, 1),
    (39.8, 1.8, 0),
    (52.0, 3.5, 0),
    (65.4, 5.2, 1),
    (78.8, 6.9, 1),
    (41.0, 8.6, 1),
    (54.4, 2.0, 0),
    (67.8, 3.7, 1),
    (30.0, 5.4, 0),
    (43.4, 7.1, 0),
    (56.8, 8.8, 1),
    (69.0, 2.2, 0),
    (32.4, 3.9, 0),
    (45.8, 5.6, 0),
    (58.0, 7.3, 1),
    (71.4, 9.0, 1),
    (34.8, 2.4, 0),
]
TEST = [
    (32, 1.6, 0),
    (39, 3.1, 0),
    (52, 3.0, 0),
    (60, 4.5, 1),
    (47, 6.5, 1),
    (70, 7.5, 1),
    (78, 8.0, 1),
    (35, 7.9, 0),
    (73, 2.5, 1),
    (57, 5.5, 1),
    (46, 4.4, 0),
```

```
    (65, 6.2, 1),
]
LABELS = ("NORMAL", "WARNING")

# Parameter scaling dihitung HANYA dari data training.
MIN_T = min(r[0] for r in TRAIN)
MAX_T = max(r[0] for r in TRAIN)
MIN_V = min(r[1] for r in TRAIN)
MAX_V = max(r[1] for r in TRAIN)
RANGE_T = MAX_T - MIN_T
RANGE_V = MAX_V - MIN_V



def predict_knn(temp, vibration, k=3, scaled=True):
    if not isinstance(k, int) or k < 1 or k > len(TRAIN) or k % 2 == 0:
        raise ValueError("Gunakan K ganjil: 1 sampai jumlah data training")
    distances = []
    for idx, (t, v, label) in enumerate(TRAIN):
        dt = temp - t
        dv = vibration - v
        if scaled:
            dt = dt / RANGE_T
            dv = dv / RANGE_V
        distance_sq = dt * dt + dv * dv
        distances.append((distance_sq, idx, label))

    # Tie jarak dipecahkan secara deterministik memakai urutan data.
    distances.sort(key=lambda r: (r[0], r[1]))
    neighbors = distances[:k]
    warning_votes = sum(r[2] for r in neighbors)
```

```
    prediction = 1 if warning_votes > k // 2 else 0
    return prediction, neighbors



def evaluate(k=3, scaled=True, verbose=False):
    correct = 0
    tp = tn = fp = fn = 0
    for temp, vib, actual in TEST:
        predicted, _ = predict_knn(temp, vib, k, scaled)
        correct += predicted == actual
        if predicted == 1 and actual == 1:
            tp += 1
        elif predicted == 0 and actual == 0:
            tn += 1
        elif predicted == 1 and actual == 0:
            fp += 1
        else:
            fn += 1
        if verbose:
            print("T=%.1f, V=%.1f | actual=%s predicted=%s" % (
                temp, vib, LABELS[actual], LABELS[predicted]))
    accuracy = 100 * correct / len(TEST)
    print("K=%d, scaled=%s | accuracy=%.2f%% (%d/%d) | TP=%d TN=%d FP=%d FN=%d"
% (
        k, scaled, accuracy, correct, len(TEST), tp, tn, fp, fn))
    return accuracy



def demo():
    print("TRAIN =", len(TRAIN), "TEST =", len(TEST))
    print("Rentang training: T=%.1f..%.1f; V=%.1f..%.1f" % (
```

```
        MIN_T, MAX_T, MIN_V, MAX_V))
    for k in (1, 3, 5, 7):
        evaluate(k)
    print("\nPengaruh scaling untuk K=3")
    evaluate(3, scaled=False)
    evaluate(3, scaled=True)



def interactive():
    print("Ketik q untuk keluar. Input sintetik, BUKAN pembacaan sensor.")
    while True:
        raw = input("Temperature [C] atau q: ").strip()
        if raw.lower() == "q":
            return
        try:
            temp = float(raw)
            vib = float(input("Vibration [mm/s]: ").strip())
            if not (20 <= temp <= 100 and 0 <= vib <= 12):
                print("Input di luar rentang simulasi (T:20..100; V:0..12)")
                continue
            k = int(input("K ganjil [1/3/5/7]: ").strip())
            pred, neighbors = predict_knn(temp, vib, k)
            print("Prediction:", LABELS[pred])
            print("Nearest neighbors (jarak kuadrat, indeks, kelas):")
            for d, idx, label in neighbors:
                print("  %.5f | %d | %s" % (d, idx, LABELS[label]))
        except (ValueError, TypeError) as e:
            print("Input tidak valid:", e)
```

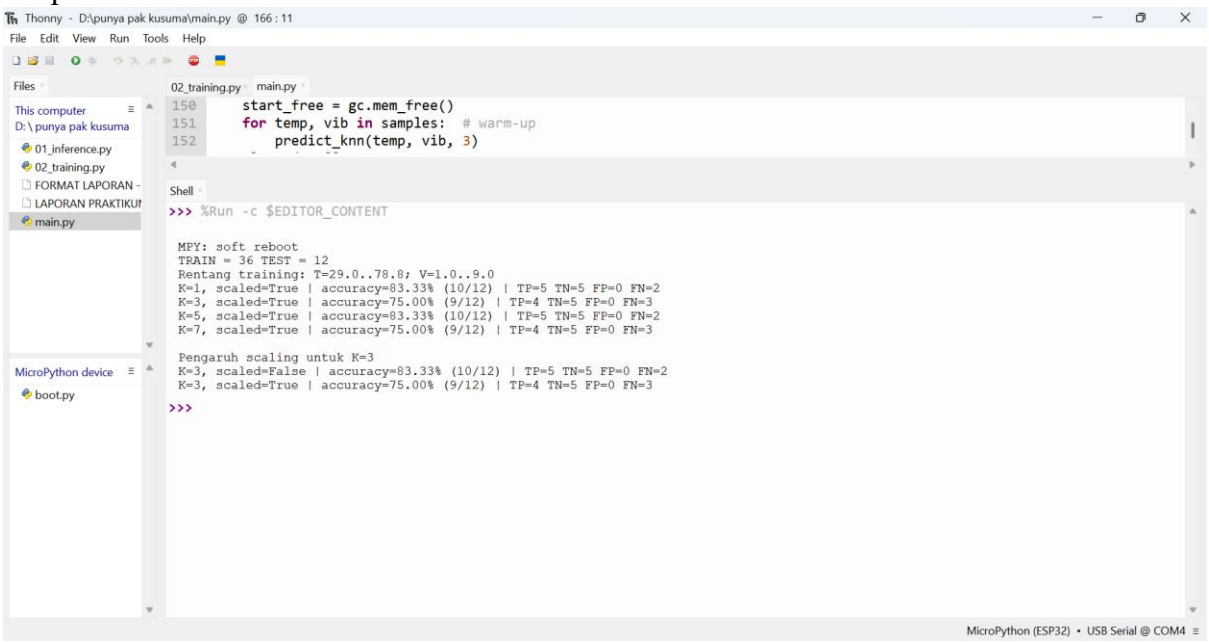
```
def benchmark(repetitions=100):
    # Menghindari waktu input/print dalam timing; tanpa dataset generator.
    samples = [(39, 3.1), (57, 5.5), (70, 7.5), (47, 6.5)]
    gc.collect()
    start_free = gc.mem_free()
    for temp, vib in samples:  # warm-up
        predict_knn(temp, vib, 3)
    elapsed = []
    for j in range(repetitions):
        temp, vib = samples[j % len(samples)]
        start = time.ticks_us()
        predict_knn(temp, vib, 3)
        elapsed.append(time.ticks_diff(time.ticks_us(), start))
    elapsed.sort()
    print("Inference median:", elapsed[len(elapsed) // 2], "us")
    print("Free memory before/after:", start_free, gc.mem_free(), "bytes")
    print("Note: these are approximate MicroPython-level measurements")



if __name__ == "__main__":
    benchmark(100)
    # Untuk eksperimen lain, panggil interactive() atau benchmark()
```

Foto/Screenshot Output Shell

a Eksperimen A



b Eksperimen B



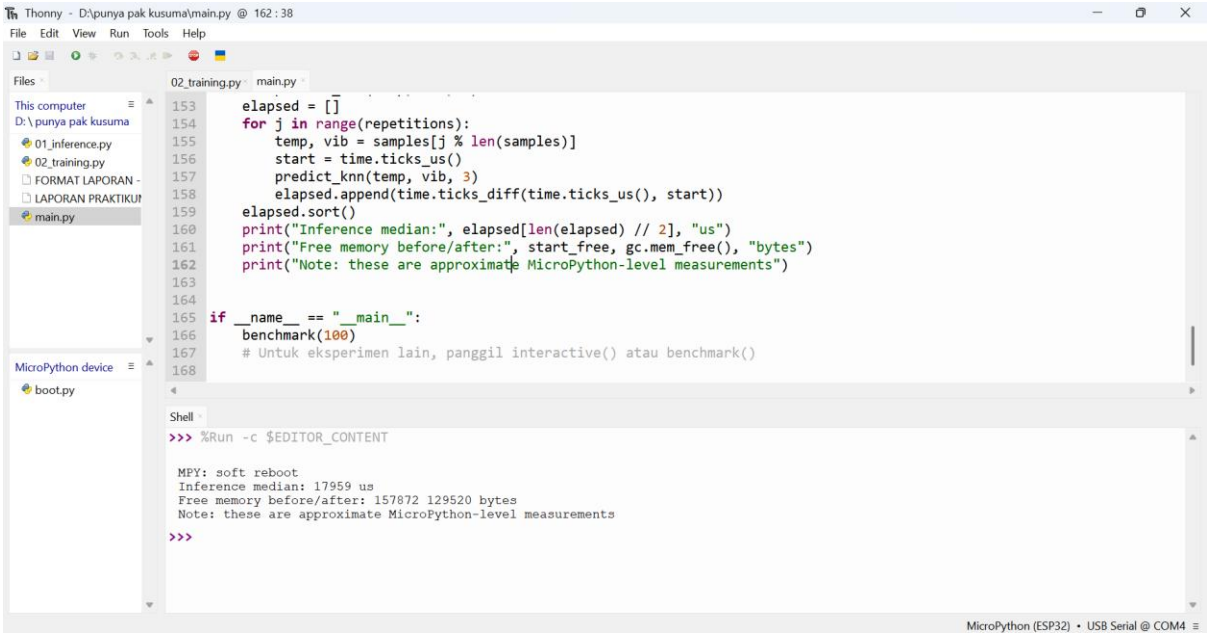
c Eksperimen C



d Eksperimen D



e Eksperimen E



Tabel Eksperimen :

Variasi	**K**	**Scaling**	**Accuracy (%)**	**Median inference (µs)**	**Catatan**
A	1	Ya	83,33%	75,00%	TP=5, TN=5, FP=0, FN=2
B	3	Ya	75,00%	75,00%	TP=4, TN=5, FP=0, FN=3
C	5	Ya	83,33%	75,00%	TP=5, TN=5, FP=0, FN=2
D	7	Ya	75,00%	75,00%	TP=4, TN=5, FP=0, FN=3
E	3	Tidak	83,33%	75,00%	Tanpa scaling (scaled=False)

Pertanyaan Analisis:

1. Mengapa KNN tidak memerlukan proses pembaruan koefisien seperti regresi linear, tetapi tetap membutuhkan data training?

   KNN adalah algoritma berbasis instance (lazy learner) yang tidak membangun model matematika parametrik (tidak mencari koefisien bobot/slope seperti OLS). Meskipun demikian, data training tetap mutlak dibutuhkan karena seluruh data tersebut disimpan mentah-mentah di dalam memori untuk digunakan sebagai titik acuan/referensi saat menghitung jarak ke data baru pada saat inference.

2. Mengapa normalisasi dapat mengubah prediksi KNN? Jelaskan menggunakan skala temperatur dan getaran.

   Tanpa normalisasi (scaling), rentang nilai fitur temperatur (misal 30–80 °C) jauh lebih besar dibandingkan getaran (misal 1–9 mm/s). Dalam rumus jarak Euclidean, selisih angka pada temperatur akan mendominasi total kuadrat jarak secara masif, sehingga fitur getaran seolah-olah tidak dianggap oleh model. Normalisasi diperlukan agar kedua fitur memiliki rentang skala yang setara (misalnya antara 0 sampai 1), sehingga kontribusi jarak dari temperatur dan getaran menjadi adil dan seimbang.

3. Apakah K yang lebih besar selalu menghasilkan accuracy lebih tinggi? Gunakan hasil percobaan.

   Tidak selalu. Berdasarkan hasil percobaan yang telah dilakukan, nilai K yang lebih besar tidak menjamin akurasi yang lebih tinggi. Akurasi sangat bergantung pada sebaran data dan struktur batas antar kelas nilai K yang terlalu besar justru dapat mengaburkan batas keputusan (boundary smoothing).

4. Apa perbedaan waktu inference dan waktu input melalui terminal serial? Mengapa *timing* tidak dilakukan saat input() atau print()?

   - Waktu inference adalah durasi waktu murni yang dibutuhkan mikrokontroler untuk melakukan komputasi algoritma (menghitung jarak Euclidean, mengurutkan, dan melakukan majority voting).

   - Waktu input serial melibatkan proses komunikasi perangkat keras, kecepatan transmisi (baud rate), dan jeda waktu manusia mengetik di terminal.

   - Timing tidak boleh dilakukan pada input() atau print() karena hal tersebut hanya mengukur kecepatan respons serial dan waktu ketik manusia, bukan performa kecepatan komputasi algoritma yang objektif.

5. Jika data training diperbesar 10 kali, apa yang kemungkinan terjadi pada penggunaan memori dan waktu inference di ESP32? Apa yang perlu diukur untuk membuktikannya?

   Jika data training diperbesar 10 kali, penggunaan memori RAM di ESP32 akan meningkat secara linier karena algoritma KNN menyimpan seluruh data latih mentah-mentah di dalam memori, dan waktu inference akan menjadi jauh lebih lambat akibat bertambahnya jumlah perhitungan jarak Euclidean yang harus diproses terhadap seluruh titik referensi (O(N)). Untuk membuktikannya secara empiris, hal yang perlu diukur adalah kapasitas memori bebas menggunakan fungsi gc.mem_free() sebelum dan sesudah data diperbesar, serta waktu eksekusi fungsi prediksi menggunakan pewaktu berbasis mikrodetik seperti time.ticks_us().